

Reviewer Pack — Minimal Index of Braided Monoid Automorphisms and Communicat...

Entry 5c0d10878f2f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 12:24:09 UTC

Minimal Index of Braided Monoid Automorphisms and Communication Complexity Rank Variance

Entry ID: 5c0d10878f2f **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 12:24:09 UTC

1. Conjecture

Field A (mathematical branch): Category Theory (Braided Monoids) **Field B** (complexity object): Communication Complexity (Matrix Rank Variance)

Statement:

For every n -bit CNF formula, the minimal index of automorphism groups of its associated braided monoid is polynomially related to its communication complexity rank variance, such that $|\min_index(\alpha)| = O(f(n))$ where $f(n) \leq C * \log^2(n)$.

Rationale (proposer's reasoning):

Braided monoids encode noncommutative algebraic structures and have been used in quantum computation. Their automorphism groups can potentially capture hidden symmetries in the communication complexity of a given problem instance.

Taxonomy category: BraidedMonoidAutomorphisms (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a23b6a551d157481

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between the minimal index of automorphism groups and the communication complexity rank variance for all n -bit CNF formulas ($n \leq 40$) meets the condition: 'correlation_coefficient ≥ 0.8 '. The conjecture is falsified if this condition is not met.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'braided monoids' AND 'communication complexity rank variance' - 'automorphism groups' INBRACKET braided monoid AND communication complexity - 'matrix rank variance' NEAR/10 'minimal index automorphisms'

Top relevant hits considered: - [http://arxiv.org/abs/1112.4250v1] The Rank and Hanna Neumann Property of Some Submonoids of a Free Monoid - [http://arxiv.org/abs/2107.13442v1] Koszulity of dual braid monoid algebras via cluster complexes - [http://arxiv.org/abs/2410.11047v2] Principal ideals in a plactic monoid always intersect - [http://arxiv.org/abs/1511.09051v3] On automorphism groups of affine surfaces - [http://arxiv.org/abs/1601.00169v5] Braided quantum groups and their bosonizations in the C^* -algebraic framework - [http://arxiv.org/abs/1010.3612v2] Explicit examples of equivalence relations and factors with prescribed fundamental group and outer automorphism group - [http://arxiv.org/abs/1901.05908v1] Locality in Index Coding for Large Min-Rank - [http://arxiv.org/abs/1906.03766v3] Variance Reduction in Gradient Exploration for Online Learning to Rank

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.2s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            clauses.append(clause)
        return clauses

    def minimal_index(m, n):
        # Placeholder function; actual implementation needed
        return random.randint(1, m)

    def communication_complexity_rank_variance(cnf):
        # Placeholder function; actual implementation needed
        return random.random()

    def compute_correlation(trials):
```

```

x = [trial['minimal_index'] for trial in trials]
y = [trial['communication_complexity_rank_variance'] for trial in trials]
n = len(x)
mean_x = sum(x) / n
mean_y = sum(y) / n
cov = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(n)) / n
var_x = sum((x[i] - mean_x)**2 for i in range(n)) / n
var_y = sum((y[i] - mean_y)**2 for i in range(n)) / n
return cov / (math.sqrt(var_x) * math.sqrt(var_y))

trials = []
for _ in range(30):
    n = random.choice([5, 10, 15, 20, 30, 40])
    cnf = generate_cnf(n)
    min_index_val = minimal_index(len(cnf), n)
    cc_variance = communication_complexity_rank_variance(cnf)
    trials.append({
        "minimal_index": min_index_val,
        "communication_complexity_rank_variance": cc_variance
    })

correlation_coefficient = compute_correlation(trials)

return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": 30,
    "n_max": max(n for _, n in trials),
    "conjecture_holds": correlation_coefficient >= 0.8,
    "counterexample": "" if correlation_coefficient >= 0.8 else "correlation_coefficient < 0.8"
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_corr_coeff = sum(res['metric_value'] for res in results) / len(results)
    support_fraction = sum(1 for res in results if res['conjecture_holds']) / len(results)

    if all(res['conjecture_holds'] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_corr_coeff} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_corr_coeff} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res['conjecture_holds'])
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient < 0.8\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
_variance', 'conjecture_holds': False, 'counterexample': 'correlation_coefficient < 0.8'}  
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': -0.29409216605224753, 'instances_t'  
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': -0.034703389222646416, 'instances_t'  
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': -0.21474873821103305, 'instances_t'  
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.19195546559067656, 'instances_t'  
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': -0.3227686687545012, 'instances_t'  
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.3842578623347649, 'instances_t'  
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': -0.21087520231577847, 'instances_t'  
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': -0.01884664089712287, 'instances_t'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code uses placeholder functions for ‘minimal_index’ and ‘communication_complexity_rank_variance’, which are replaced with random values. This does not provide a meaningful empirical test of the conjecture, as it lacks a proper implementation of the required primitives.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The correlation coefficient between the minimal index of automorphism groups and the communication complexity rank variance did not meet the pre-regis | next: Investigate the relationship between the minimal index of automorphism groups and the communication complexity rank variance further using a proper implementation of the required primitives.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14280
2	preregistration	ollama_remote	glm4:latest	0	0	15867
3	novelty	ollama_remote	glm4:latest	0	0	8308
4	novelty	ollama_remote	glm4:latest	0	0	9706
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13312
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10572
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10235
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10150
9	critic	ollama_remote	glm4:latest	0	0	14047
10	judge	ollama_remote	glm4:latest	0	0	9485

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 115962 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in [research/audit/5c0d10878f2f.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5c0d10878f2f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/5c0d10878f2f.tar.gz` (if generated)